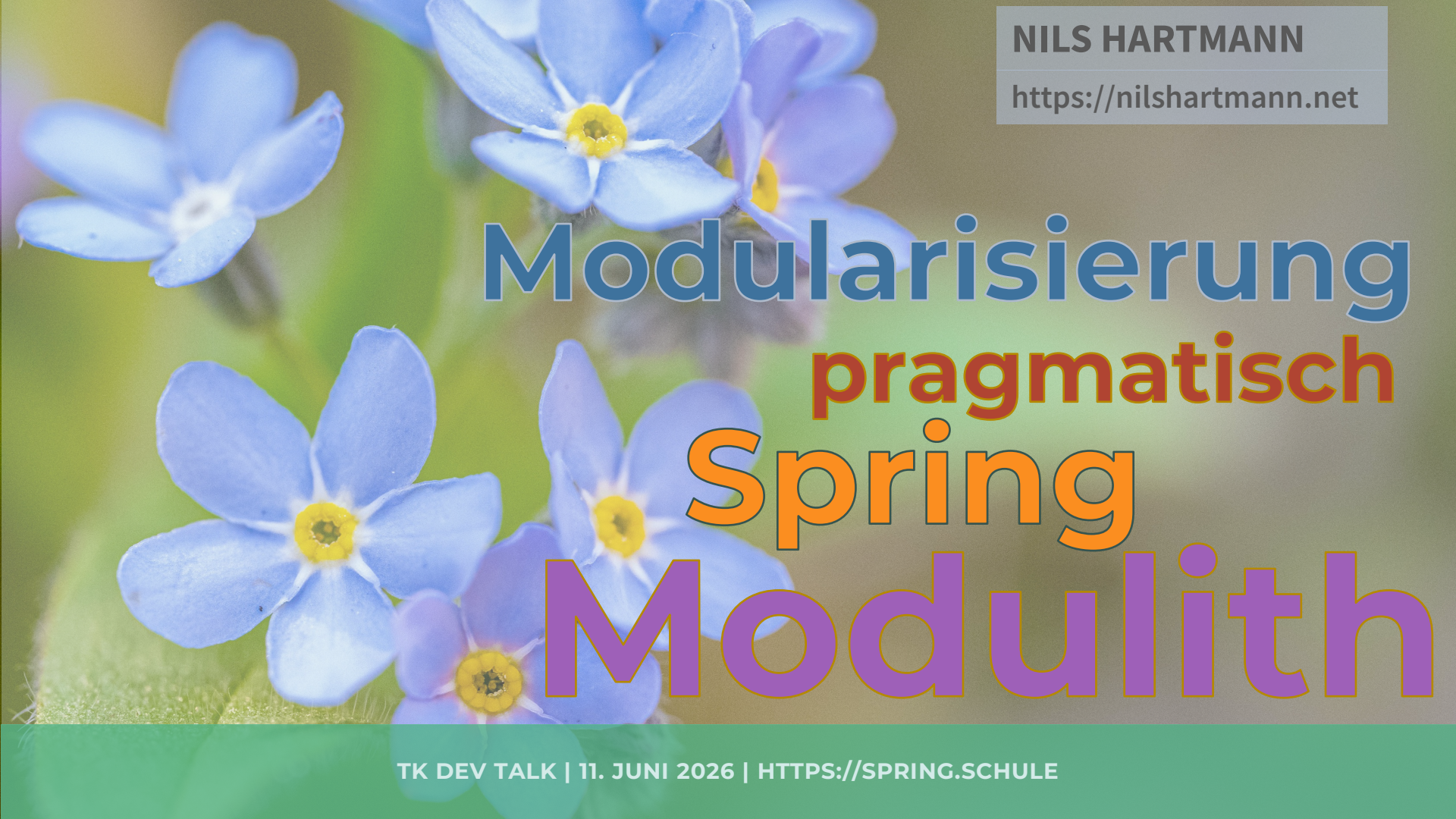




NILS HARTMANN

<https://nilshartmann.net>

Modularisierung pragmatisch Spring Modulith

TK DEV TALK | 11. JUNI 2026 | [HTTPS://SPRING.SCHULE](https://spring.schule)

NILS HARTMANN

nils@nilshartmann.net

Freiberuflicher Software-Entwickler, –Architekt, Coach, Trainer

Java, Spring, GraphQL, React, TypeScript



<https://react.schule/video-kurs-react>



<https://reactbuch.de>

[HTTPS://NILSHARTMANN.NET](https://nilshartmann.net)

NILS HARTMANN

„Früher“



ca. 2006 (nicht nachkoloriert)



<https://nilshartmann.net/osgi>

[HTTPS://NILSHARTMANN.NET](https://nilshartmann.net)



Q Search

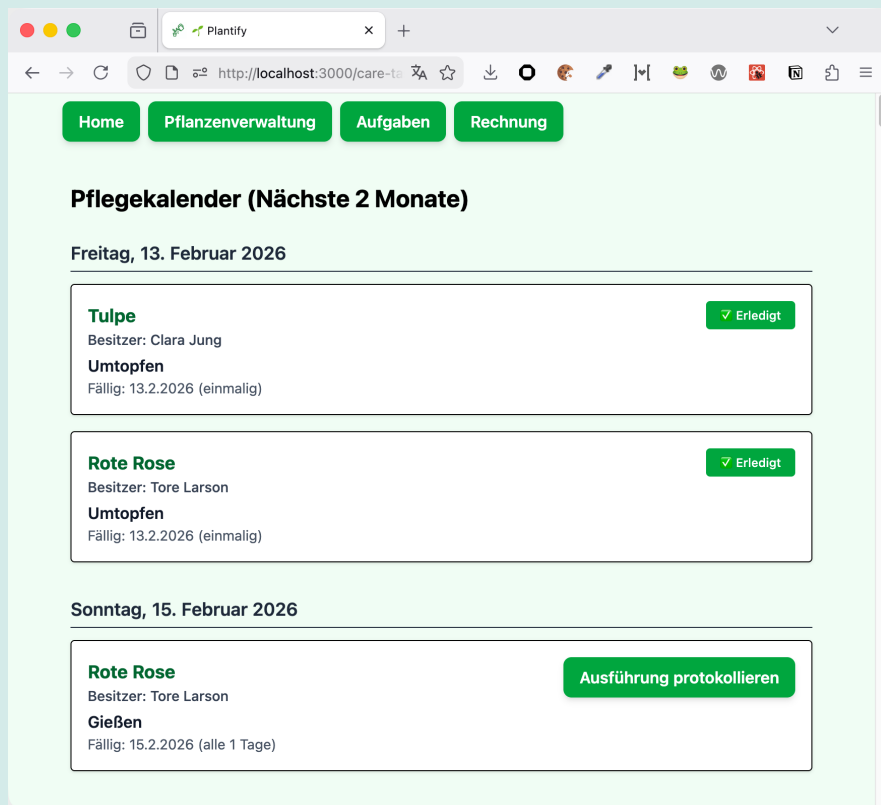
⌘ + k



Fundamentals

Spring Modulith supports developers implementing logical **modules** in Spring Boot applications. It allows them to apply structural **validation, document** the module arrangement, run integration tests for individual modules, observe the modules' interaction at runtime, and generally implement module interaction in a **loosely coupled** way.

<https://docs.spring.io/spring-modulith/reference/fundamentals.html>

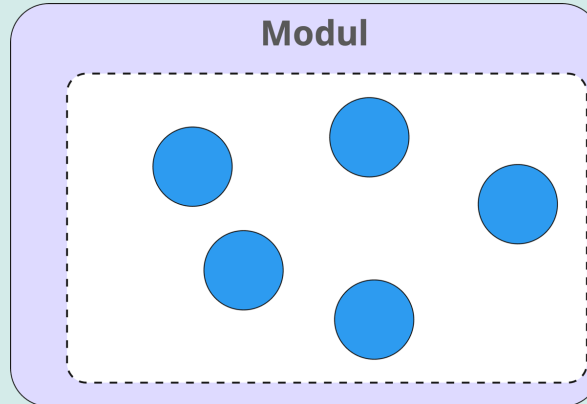


BEISPIEL

MODULE

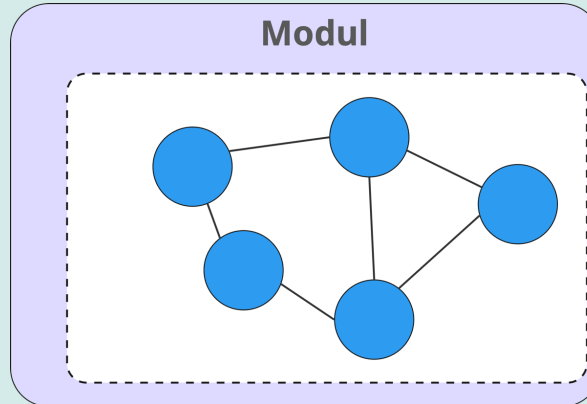
- Klare fachliche Verantwortung, in-sich abgeschlossen

-



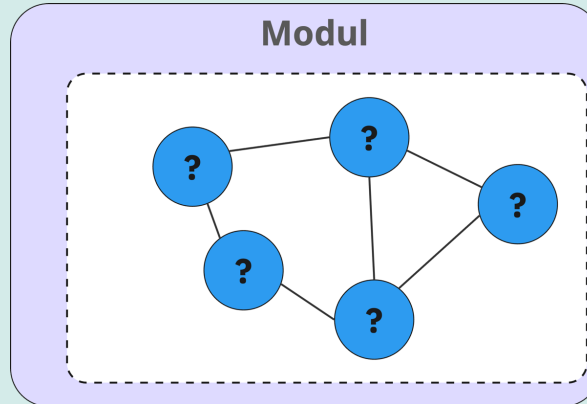
MODULE

- Klare fachliche Verantwortung, in-sich abgeschlossen
- Hohe Kohäsion innerhalb



MODULE

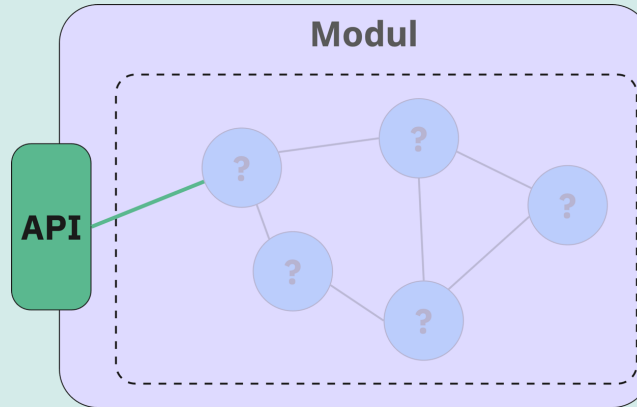
- Klare fachliche Verantwortung, in-sich abgeschlossen
- Hohe Kohäsion innerhalb
- Innenleben bleibt verborgen



MODULE

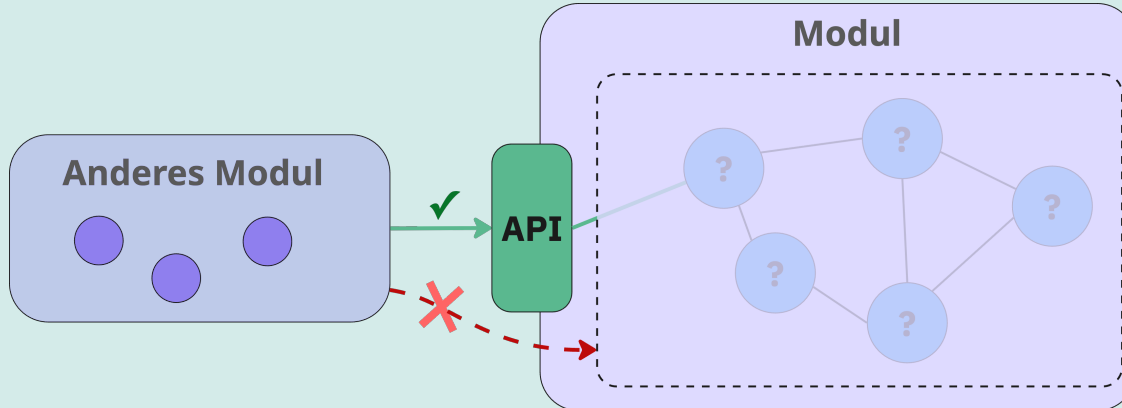
- Nach außen: explizite Schnittstelle

-



MODULE

- Nach außen: explizite Schnittstelle
- Lose Kopplung, geringe Kohäsion
-

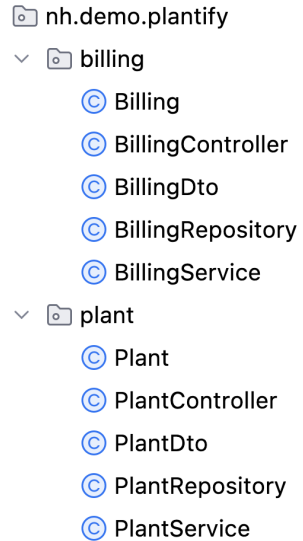


- **Microservices – sehr konsequenter Ansatz** 😊
 - Eigene Codebasis, Programmiersprache etc. wählbar
 - Unabhängig deploybar
 - Eigene Datenhaltung
 - Definierte Schnittstelle (REST API, Kafka, ...)
- **Microservices – sehr teurer Ansatz** 💰
 - Konsistenz / Transaktionen
 - Latenz
 - Betrieb, Operations, Monitoring

- **Microservices – sehr konsequenter Ansatz** 😊
- Eigene Codebasis, Programmiersprache etc. wählbar
- Unabhängig deploybar
- Eigene Datenhaltung
- Definierte Schnittstelle (REST API, Kafka, ...)

Modularisierung mit Java-Bordmitteln?

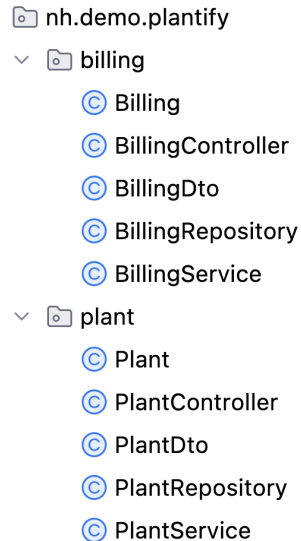
Modularisierung mit Java-Bordmitteln?



```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
└── plant
    ├── Plant
    ├── PlantController
    ├── PlantDto
    ├── PlantRepository
    └── PlantService
```

The screenshot shows a package structure in an IDE. The root package is `nh.demo.plantify`. It contains two sub-packages: `billing` and `plant`. The `billing` package contains five classes: `Billing`, `BillingController`, `BillingDto`, `BillingRepository`, and `BillingService`. The `plant` package contains five classes: `Plant`, `PlantController`, `PlantDto`, `PlantRepository`, and `PlantService`. Each class is represented by a blue circle icon with a 'C' inside.

Gliederung der Anwendung in fachliche Module mit Packages

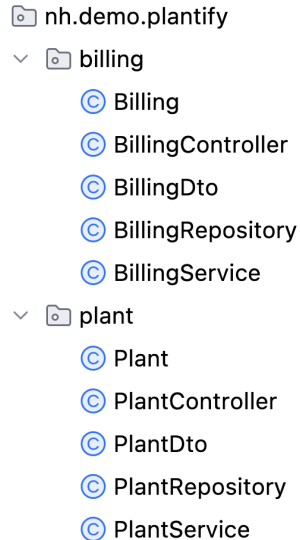


```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
└── plant
    ├── Plant
    ├── PlantController
    ├── PlantDto
    ├── PlantRepository
    └── PlantService
```

The screenshot shows a file explorer view of a Java project. The root package is 'nh.demo.plantify'. It contains two sub-packages: 'billing' and 'plant'. The 'billing' package contains five classes: 'Billing', 'BillingController', 'BillingDto', 'BillingRepository', and 'BillingService'. The 'plant' package contains five classes: 'Plant', 'PlantController', 'PlantDto', 'PlantRepository', and 'PlantService'. Each class is represented by a blue circle icon.

- Fachlichkeit ist in **einem** Package konzentriert

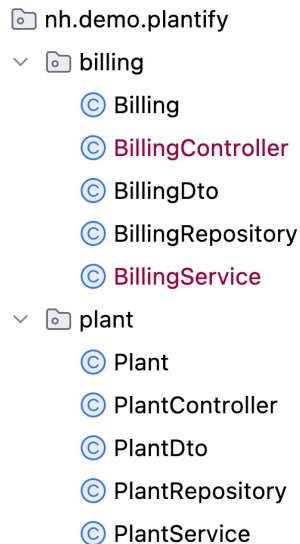
Gliederung der Anwendung in fachliche Module mit Packages



```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
└── plant
    ├── Plant
    ├── PlantController
    ├── PlantDto
    ├── PlantRepository
    └── PlantService
```

- Fachlichkeit ist in **einem** Package konzentriert
- Vertikaler „**Slice**“, enthält alle Schichten

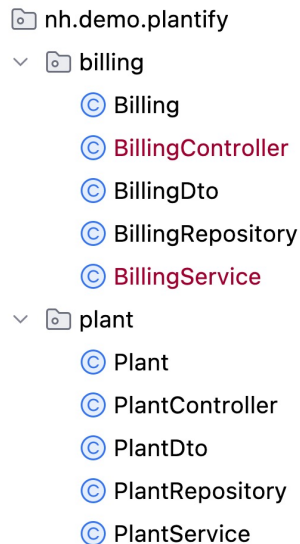
Gliederung der Anwendung in fachliche Module mit Packages



```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
└── plant
    ├── Plant
    ├── PlantController
    ├── PlantDto
    ├── PlantRepository
    └── PlantService
```

- Fachlichkeit ist in **einem** Package konzentriert
- Vertikaler „**Slice**“, enthält alle Schichten
- **Änderungen** müssen nur an **einer Stelle** gemacht werden

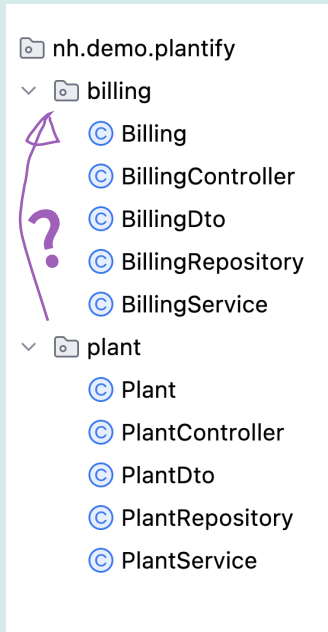
Gliederung der Anwendung in fachliche Module mit Packages



```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
└── plant
    ├── Plant
    ├── PlantController
    ├── PlantDto
    ├── PlantRepository
    └── PlantService
```

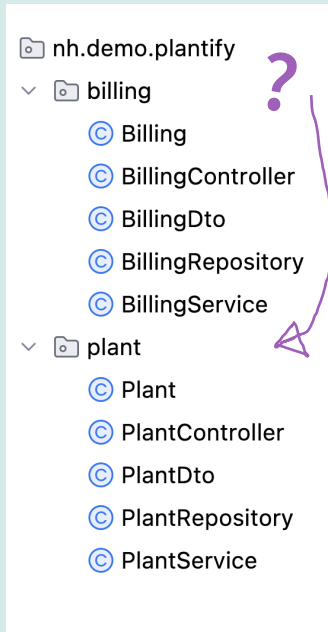
- Fachlichkeit ist in **einem** Package konzentriert
- Vertikaler „**Slice**“, enthält alle Schichten
- **Änderungen** müssen nur an **einer Stelle** gemacht werden
- Hohe Kohäsion

Gliederung der Anwendung in fachliche Module mit Packages



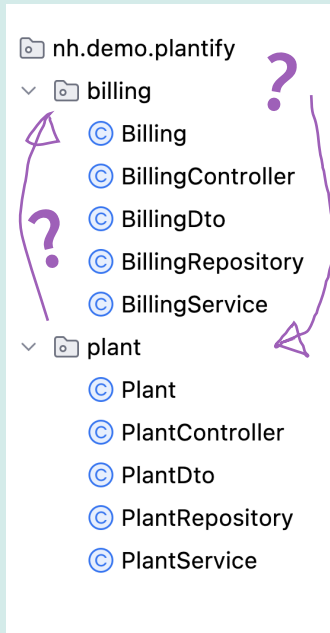
- Problem: Wie sehen Abhängigkeiten aus?

Gliederung der Anwendung in fachliche Module mit Packages



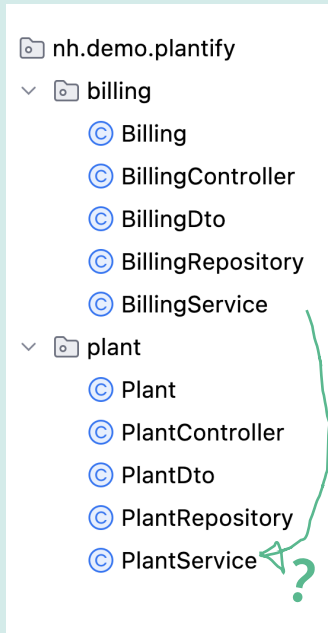
- Problem: Wie sehen Abhängigkeiten aus?
- Oder so?

Gliederung der Anwendung in fachliche Module mit Packages



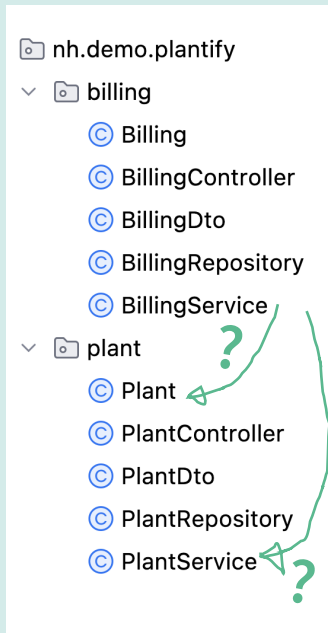
- Problem: Wie sehen Abhängigkeiten aus?
- Oder so?
- Mal so, mal so 🤔

Gliederung der Anwendung in fachliche Module mit Packages



- Problem: Wie sieht die **Schnittstelle** von „plant“ aus?
- 🤔

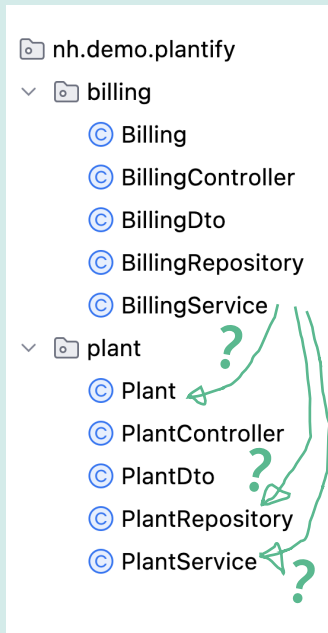
Gliederung der Anwendung in fachliche Module mit Packages



- Problem: Wie sieht die **Schnittstelle** von „plant“ aus?



Gliederung der Anwendung in fachliche Module mit Packages

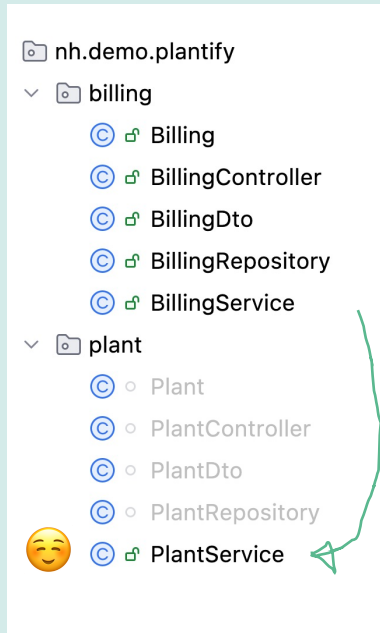


• Problem: Wie sieht die **Schnittstelle** von „plant“ aus?



Gliederung der Anwendung in fachliche Module mit Packages

- Problem: Wie sieht die **Schnittstelle** von „plant“ aus?



Sichtbarkeit von Klassen einschränken

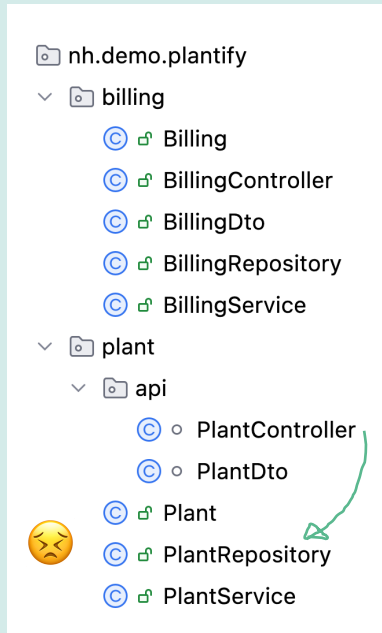
```
package nh.demo.plantify.plant;  
  
// Öffentlich  
public class PlantService {  
  
    // ...  
  
}
```



```
package nh.demo.plantify.plant;  
  
// Nur Plant-intern  
class PlantRepository {  
  
    // ...  
  
}
```



Gliederung der Anwendung in fachliche Module mit Packages



Sichtbarkeit von Klassen einschränken

```
package nh.demo.plantify.plant;

// Öffentlich
public class PlantService {

    // ...

}
```



```
package nh.demo.plantify.plant;

// Nur Plant-intern
public class PlantRepository {

    // ...

}
```

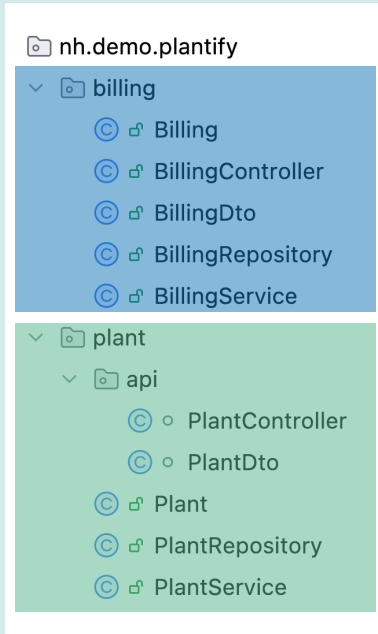


Modularisierung mit Java-Bordmitteln

```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
├── plant
│   └── api
│       ├── PlantController
│       ├── PlantDto
│       ├── Plant
│       ├── PlantRepository
│       └── PlantService
```

Was fehlt?

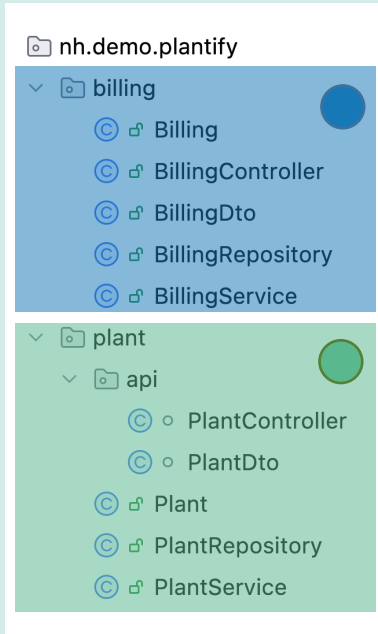
Modularisierung mit Java-Bordmitteln



Was fehlt?

- „Echte“ Module
 - Mehr als Klassen oder Packages

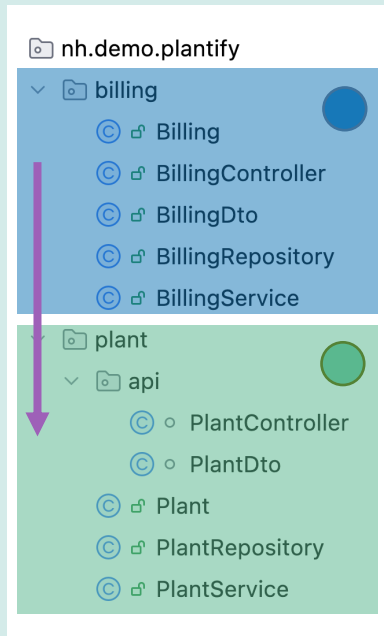
Modularisierung mit Java-Bordmitteln



Was fehlt?

- „Echte“ Module
 - Mehr als Klassen oder Packages
- Klare Schnittstellen
 - ...

Modularisierung mit Java-Bordmitteln

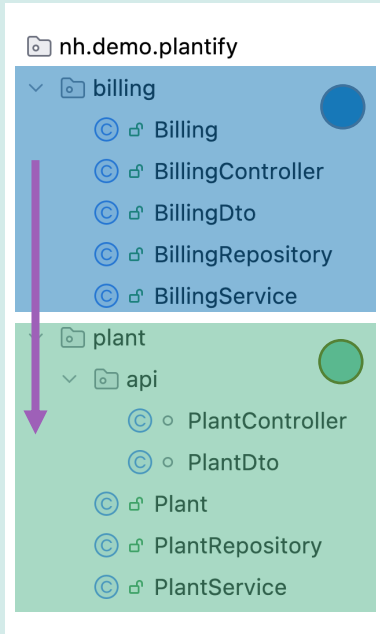


Was fehlt?

- „Echte“ Module
 - Mehr als Klassen oder Packages
- Klare Schnittstellen
 - Inneres verbergen
- Explizite Abhängigkeiten
 - Zwischen den Modulen

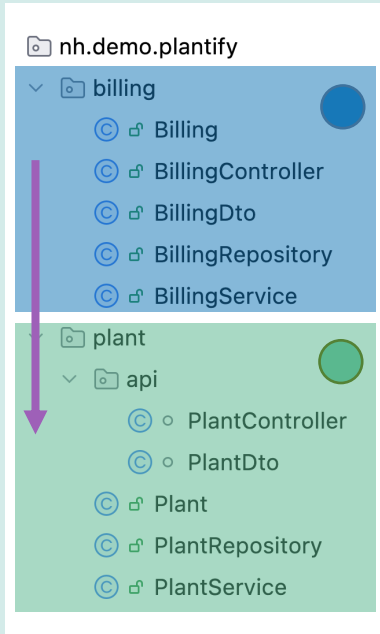
SPRING MODULITH

- Spring Modulith

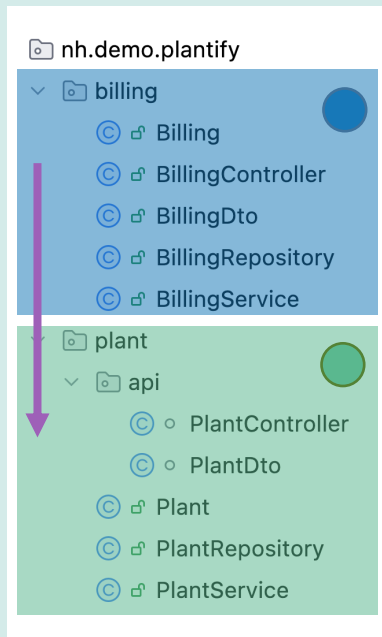


- Spring Modulith

- Packages werden zu *Application Modules*

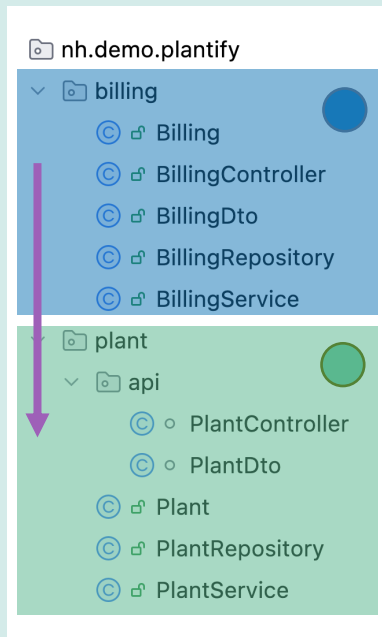


- Spring Modulith



- Packages werden zu *Application Modules*
 - Direkte Packages unterhalb des main-Packages
 - Definierte Abhängigkeiten
 - Explizite Schnittstellen

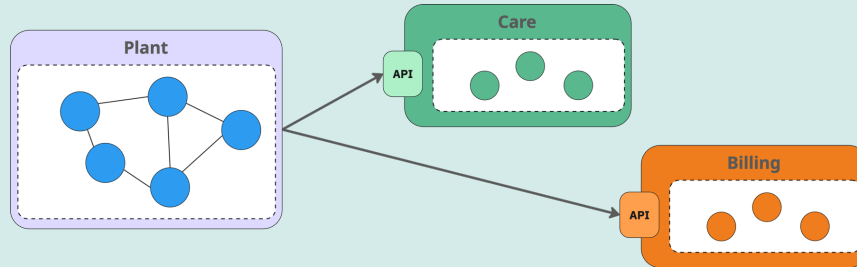
- Spring Modulith



- Packages werden zu *Application Modules*
 - Direkte Packages unterhalb des main-Packages
 - Definierte Abhängigkeiten
 - Explizite Schnittstellen

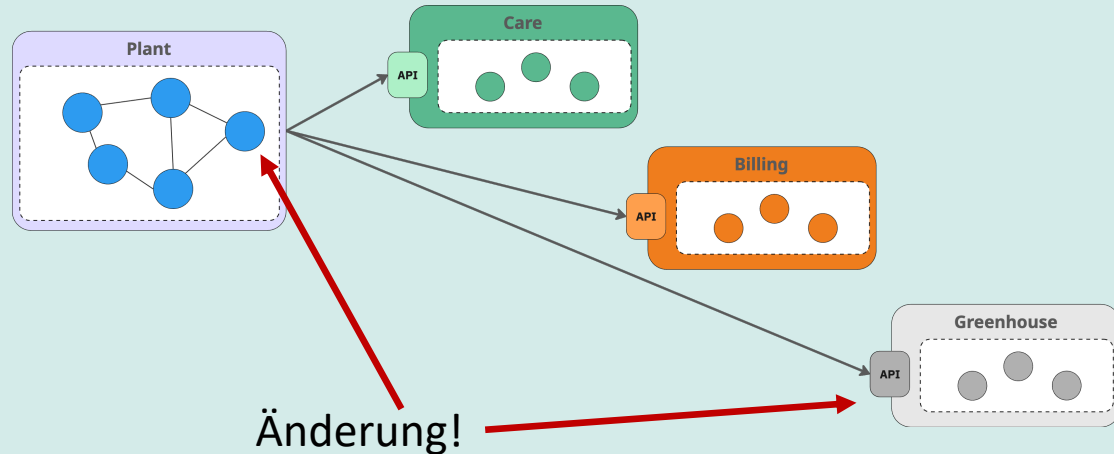


Kopplung von Modulen über Services



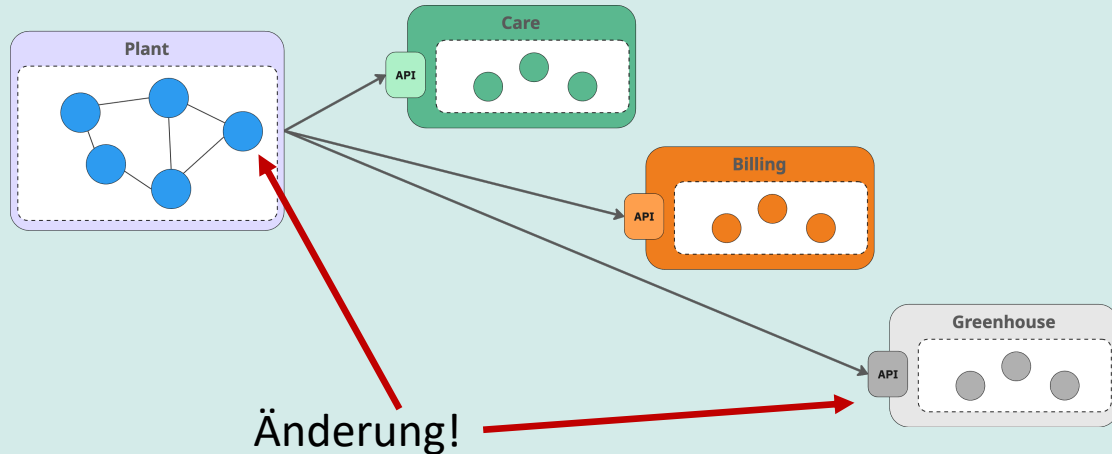
Kopplung von Modulen über Services

- Erweiterungen müssen an zwei Stellen erfolgen
- Tests müssen angepasst werden (Beans in Services!)



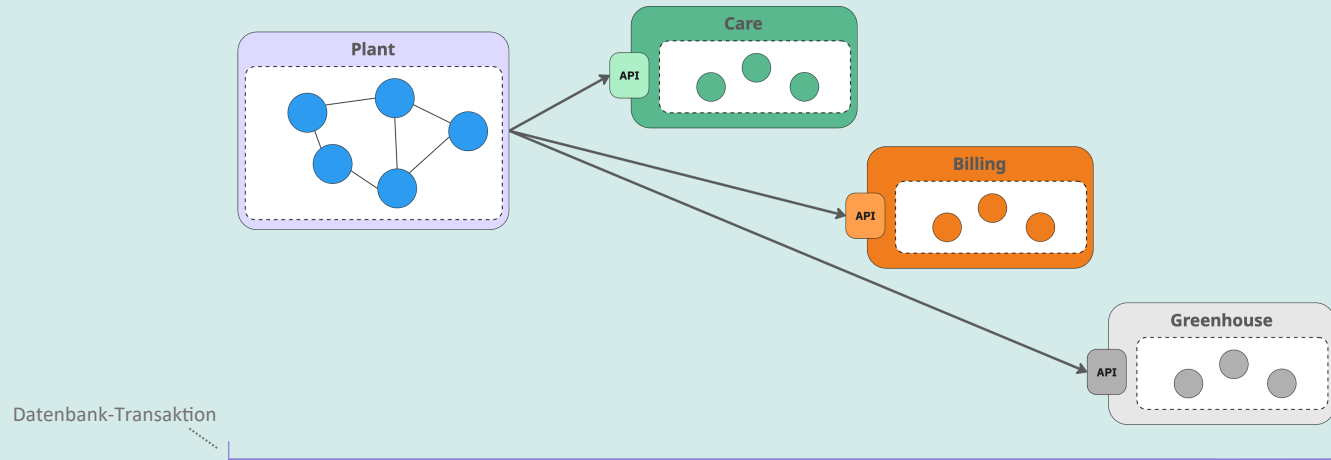
Kopplung von Modulen über Services

- Erweiterungen müssen an zwei Stellen erfolgen
- Tests müssen angepasst werden (Beans in Services!)
- Verletzung des Open-Closed-Prinzips



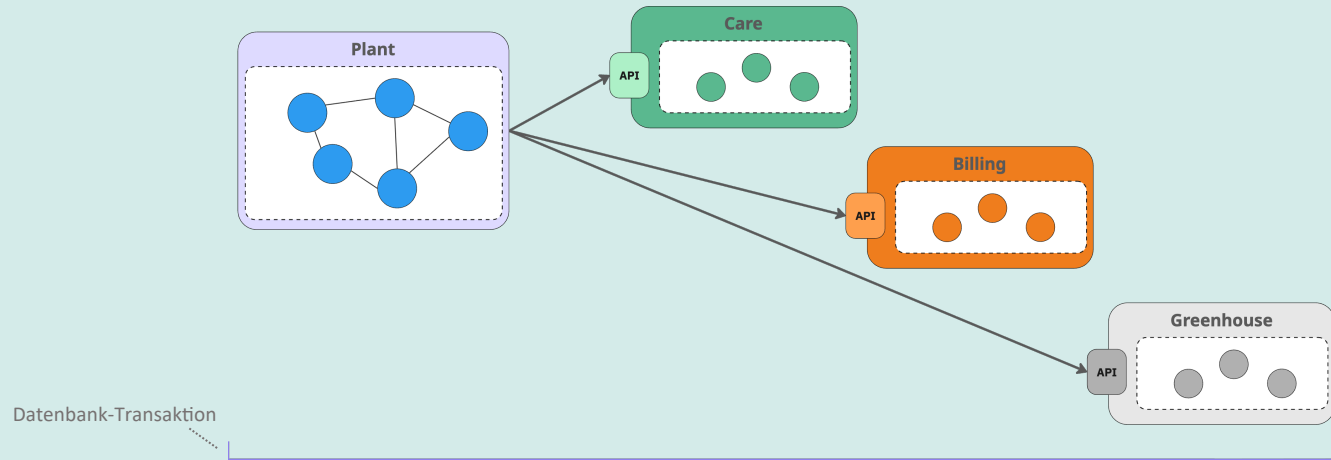
Datenbank-Transaktion

- Komplette Verarbeitung in einer Transaktion
- Garantiert Konsistenz



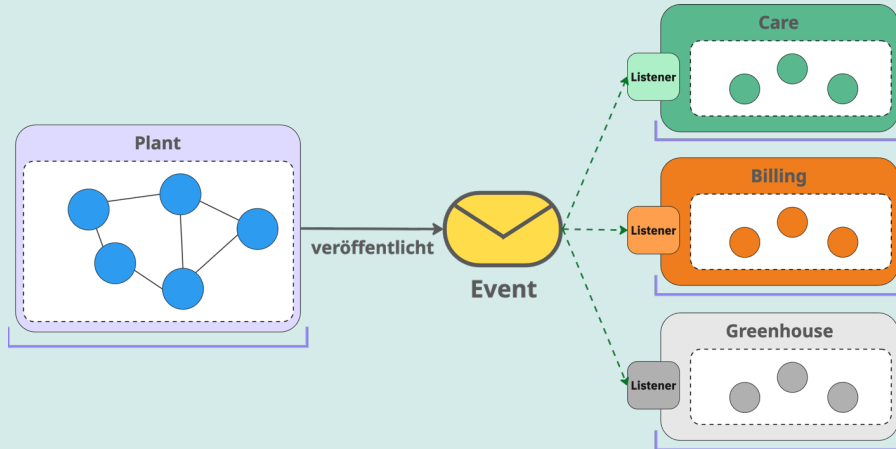
Datenbank-Transaktion

- Komplette Verarbeitung in einer Transaktion
- Garantiert Konsistenz
- Kann aber Ressourcen-Problem werden



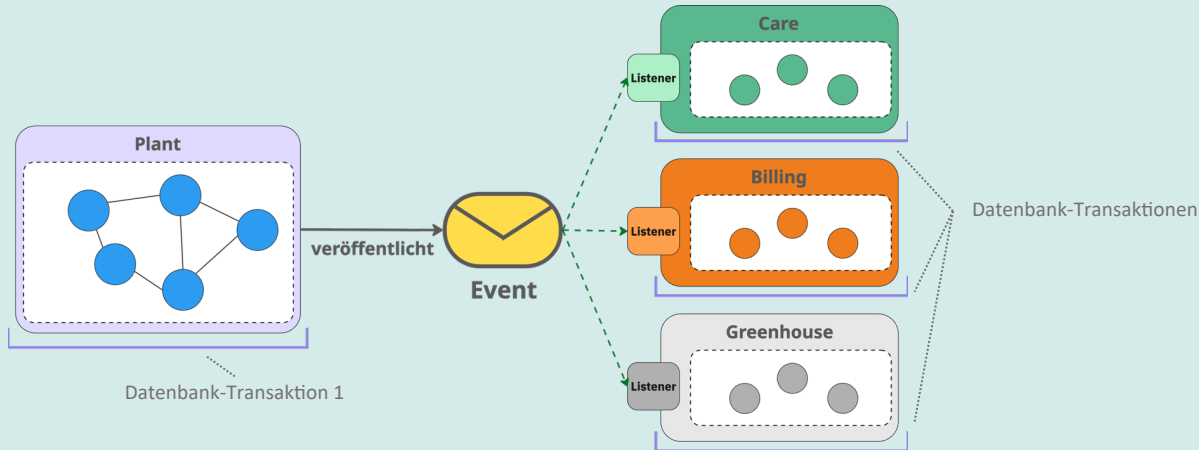
Entkopplung über Events

- Module veröffentlichen fachliche Events (Spring built-in Event Registry)
- Neue Consumer können jederzeit hinzugefügt werden



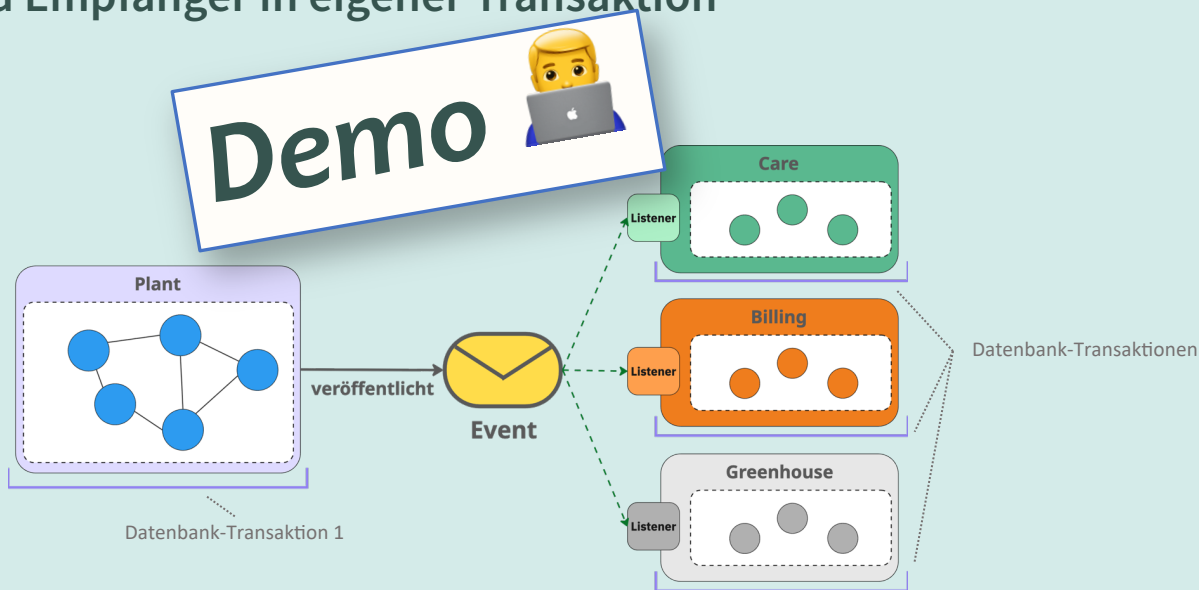
Entkopplung über Events

- Abarbeitung kann asynchron und parallel erfolgen
- Sender und Empfänger in eigener Transaktion



Entkopplung über Events

- Abarbeitung kann asynchron und parallel erfolgen
- Sender und Empfänger in eigener Transaktion



NILS HARTMANN

<https://nilshartmann.net>

Vielen Dank!

Slides & Code: <https://spring.schule/tk2026-spring-modulith>

Fragen und Kontakt:

nils@nilshartmann.net

<https://nilshartmann.net/kontakt>

